IEEE 829 Test Plan

SDNScope

Socket-Based Network Diagnostic and Path Analysis Tool for Software-Defined Networks

Prepared by
Sudhanva Y K, Suhana A Ghosh, Siddhanth S

Computer Networks Project (cn_project)

November 2026

Test plan structure follows the IEEE 829 software test documentation standard.

1. Test Plan Identifier

- **Test Plan ID:** TP-SDNS-001
- **Project:** SDNScope – SDN diagnostic and path analysis tool
- **Version:** 1.1
- **Date:** 5 November 2026
- **Prepared by:** Sudhanva Y K, Suhana A Ghosh, Siddhanth S

2. References

1. `software_requirements_specification_for_sdnscope.tex` – the requirements this plan verifies.
2. `docs/DESIGN.md` – architecture and internal interfaces.
3. `docs/schemas.md` – JSON formats used by the tests.
4. `TEST_CASES.md` – the test case catalogue.
5. `TESTER_REPORT.md` and `BUG_REPORT.md` – the tester's results and the defect found.
6. `docs/EVIDENCE.md` and `TEST_EVIDENCE/` – captured transcripts and output files.
7. IEEE 829, *IEEE Standard for Software Test Documentation*.
8. Project repository: `https://github.com/sudhanvayk/cn_project`.

3. Introduction

This document explains how SDNScope is tested and what counts as a pass. It covers two environments: the offline localhost demo, which needs no root access, and the live Mininet + Ryu testbed, which does.

The goal is simple: prove that the tool measures delay and loss correctly, discovers the right topology, finds the right failed link, installs a working backup path, and produces readable evidence. The plan also records what is deliberately out of scope, so that nobody mistakes a skipped check for a passed one.

4. Test Items

The following items are tested:

- UDP probe server (`probe_server.py`).
- UDP probe client and statistics (`diagnostic_client.py`).
- IP-layer trace and parser (`path_analysis.py`).
- Controller topology discovery (`sdn_topology.py`).
- Graph model and path ranking (`topology_graph.py`).
- Flow-table path reconstruction (`flow_inspector.py`).
- Correlation, localization, and backup ranking (`correlate.py`).
- Reroute rule generation and deployment (`reroute_helper.py`).
- Report generation (`report_generator.py`).
- Unified CLI (`sdnscope.py`).
- Mock controller and localhost demo (`mock_controller.py`, `scripts/demo_localhost.sh`).
- Live scenario runner and cleanup (`topology/scenario_runner.py`, `scripts/run_all_scenarios.sh`).

5. Software Risk Issues

The main risks during testing are:

- **Missing privileges.** Mininet and `tc` need root. If sudo is not available, the live scenarios cannot run and must be marked `NOT RUN`, not passed.
- **Missing Ryu.** The controller must be installed and able to bind ports 6653 and 8080. A missing Ryu stops the live run at the start.
- **Leftover state.** A failed run can leave `tc` rules, OpenFlow entries, or background servers behind and distort the next run. Cleanup is therefore part of the test.
- **OpenFlow counters do not show qdisc drops.** Packets dropped by `netem` are not counted in the switch port drop counters. A test that only looks at OpenFlow would wrongly report zero drops. The qdisc capture exists to cover this gap.
- **Mock and live behaviour can differ.** The mock controller is useful, but it is not proof of switch behaviour. Live results are reported separately.
- **UDP is lossy by nature.** A single lost probe is not a failure. The tests use enough probes and a defined threshold so that the verdict is stable.
- **Timing sensitivity.** A very short outage may start and end between controller polls. The tool must fall back to probe evidence instead of claiming a link event it did not see.

6. Features to be Tested

- Installing the project and its dependencies.
- UDP probing, RTT statistics, jitter, and packet-loss measurement.
- Artificial delay, partial loss, and total loss using the probe server.
- Topology discovery from the mock controller and from Ryu.
- Path enumeration for the diamond, including the primary and backup routes.
- Flow-table reconstruction of the active path.
- Health classification and failure localization.
- Reroute rule generation, dry run, and live installation.
- Markdown, HTML, and PNG report generation.
- Input validation and error handling.
- Repeatability and cleanup after a run.
- The complete four-scenario live run: healthy, degraded, failed, and recovery.

7. Features not to be Tested

- The internal code of Ryu, Open vSwitch, Mininet, or the Linux kernel.
- Real physical switches, real routers, or a production network.
- IPv6, which the current release does not support.
- Multi-controller setups and topologies larger than the three-switch diamond.
- Long-duration soak testing or high-load stress testing.
- Security penetration testing beyond input validation and local binding.

8. Approach

Testing is done in four layers, from smallest to largest.

1. **Unit tests.** Each parser, statistic, graph function, and rule generator is tested with fixed inputs. These tests are fast and do not need a network.
2. **Integration tests.** The probe server and client are run together over loopback. The mock controller is used to test discovery, correlation, and reroute generation.
3. **System tests.** `make demo` runs the complete localhost workflow in one command: topology, paths, probe, trace, correlation, reroute, and report.
4. **Live system tests.** `make real-check` checks the environment, then `make real-scenarios` runs the four scenarios on Mininet with Ryu and captures the evidence.

The automated suite is run with `make test`. Static analysis is run with `make lint`. Live results are accepted only when the environment file, the probe JSON, the correlation report, and the reports are all present in the run directory.

9. Item Pass/Fail Criteria

A test case passes when the actual output matches the expected output and the supporting artifact exists. A test case fails when the output differs, when the command exits with an unexpected error, or when the expected evidence is missing.

`NOT RUN` is a separate status. It means the environment could not support the test. It is never counted as a pass. For example, if Ryu is not installed, the live scenario tests are `NOT RUN`; they become pass only after the live run has actually completed.

10. Suspension Criteria and Resumption Requirements

Testing is suspended if:

- Mininet, Open vSwitch, or Ryu cannot start, or the required ports are already in use.
- A scenario leaves `tc` rules or OpenFlow entries behind after cleanup.
- A crash prevents the JSON result from being written.
- The machine runs out of disk space or memory during a live run.

Testing resumes after the cause is fixed and the environment has been checked again with `make real-check`. Any run that was interrupted is discarded rather than reported as a partial pass.

11. Test Deliverables

- This test plan.
- The test case catalogue (`TEST_CASES.md`).
- The tester's report (`TESTER_REPORT.md`).
- The bug report (`BUG_REPORT.md`).
- The evidence index and captured files (`TEST_EVIDENCE/`).
- The four-scenario live results in `results/real_20261105_222258/` and the earlier repeat in `results/real_20261105_150848/`.
- The live CLI transcript (`TEST_EVIDENCE/live_cli_verification_20261105.txt`).
- The final test summary, which is Section 19 of this document.

12. Remaining Test Tasks

The current release is tested for the diamond topology and the documented scenarios. The following work remains for later releases:

- Regression testing after any change to the correlation rules or the reroute helper.
- IPv6 testing, once IPv6 support is added.
- Larger and multi-controller topologies.
- A longer soak test to check memory and file-handle behaviour.
- Per-segment probing, once responders are available.

13. Environmental Needs

Hardware

- A Linux machine or virtual machine with at least 4 GB of RAM. The reference live runs used an Ubuntu host with a working Mininet installation.
- A terminal with sudo access for Mininet and `tc`.

Software

- Ubuntu 22.04, 24.04, or 26.04 (the latest live run used Ubuntu 26.04).
- Python 3.10 to 3.14. The latest live run used Python 3.14.4; CI covers the full range.
- Mininet 2.3.0.
- Open vSwitch 3.7.1.
- Ryu 4.34, started with `ryu.app.ofctl_rest`, `ryu.app.rest_topology`, and `ryu.app.simple_switch_13`.
- `iproute2` for `tc qdisc netem`.
- The Python packages in `requirements.txt` and `requirements-dev.txt`.

Ports

- TCP 8080 for the Ryu REST API.
- TCP 6653 for OpenFlow (6633 is also accepted).
- UDP 9999 for the probe service.
- TCP 8181 only if OpenDaylight is used; it is not used in the default test setup.

14. Staffing and Training Needs

The work was divided between two developers and one tester:

- The developers wrote the modules, scripts, tests, and documentation, and fixed defects found during testing.
- The tester installed the project from scratch, ran the automated and manual cases, captured the evidence, and wrote the tester report and bug report.

A tester needs basic Linux command-line skills and enough Python to read an error message. Understanding Mininet and OpenFlow helps with the live scenarios, but the offline demo can be run without that knowledge.

15. Responsibilities

- **Developers:** implement the features, keep the tests green, and fix reported defects.
- **Tester:** design and execute the test cases, record actual results, and keep the evidence files.
- **Project guide or instructor:** review the plan, check the evidence, and approve the final result.

16. Schedule

Task	Date	Duration
Requirements and first implementation	19 Oct 2026	6 days
Live topology verification and first full run	25 Oct 2026	1 day
Test case design and offline test execution	2 Nov 2026	2 days
Bug fix and regression check	2 Nov 2026	1 day
Live four-scenario validation and evidence capture	5 Nov 2026	1 day
Final review of documents and evidence	5 Nov 2026	1 day

17. Planning Risks and Contingencies

- **No Ryu or no sudo.** The offline demo and the 82-test suite still run. The live cases are marked `NOT RUN` and completed later on a machine that has the tools.
- **Flaky packet loss.** The probes use a fixed count and a timeout, and the verdict uses thresholds rather than a single packet. A run that looks unstable is repeated and the repeat is kept as evidence.
- **Large result folders.** The run directories are kept small by storing text and JSON outputs; log files are excluded from version control.
- **Machine differences.** The environment file in each run records the OS, Python, Mininet, OVS, and Ryu versions, so a result can be compared fairly with another run.

18. Test Case Specification

The table below lists the cases that were executed. The “Actual result” column is copied from the captured evidence, not invented.

The tester’s first pass on 2 November recorded the live case as `NOT RUN` because Ryu was not installed in that environment. The case was executed later, on 5 November, and that later run is the evidence used here.

ID	Description	Expected result	Actual result	Status
ENV-01	Run `make install` on a clean checkout.	Virtual environment created and all packages installed.	Completed with Python 3.12.7 in the tester run and 3.14.4 in the live run.	PASS
AUTO-01	Run `make test`.	All unit and integration tests pass.	82 passed, 0 failed, 68% total coverage.	PASS
AUTO-02	Run `make lint`.	Ruff reports no findings.	`All checks passed!`	PASS
AUTO-03	Compile the project sources.	No syntax errors.	Compilation exited with code 0.	PASS
DEMO-01	Run `make run` twice.	The full localhost workflow succeeds both times.	Both runs completed: topology, paths, probe, trace, correlation, reroute, and reports.	PASS
TOPO-01	Discover the mock controller's healthy topology.	5 nodes and 5 links.	5 nodes and 5 links.	PASS
PATH-01	Enumerate the paths from h1 to h2.	Primary 3-hop and backup 4-hop paths.	`h1-s1-s2-h2` and `h1-s1-s3-s2-h2`.	PASS
UDP-01	Send 10 normal probes over loopback.	Replies received and RTT reported.	10 sent, 10 received, 0% loss.	PASS
UDP-02	Send 5 probes with a 30 ms server delay.	Mean RTT close to or above 30 ms.	5 of 5 received; mean RTT 30.696 ms.	PASS
UDP-03	Send 20 probes with a 40% drop rate.	Non-zero measured loss.	11 received, 9 lost, 45% measured loss.	PASS
UDP-04	Send 3 burst probes with a 100% drop rate.	100% loss and no RTT value.	0 received, 3 lost, 100% loss, no RTT metrics.	PASS
COR-01	Correlate a full-loss probe with the failed mock topology.	Failed path and the missing primary link identified.	`FAILED_PATH`; `s1-s2` at 100% from topology liveness.	PASS
ROUTE-01	Generate a reroute for the failed path.	Rules use the real destination address and are accepted.	12 IPv4 and ARP rules submitted; targets use 10.0.0.2 after the fix.	PASS
REPORT-01	Generate the localhost reports.	Valid JSON, Markdown, HTML, and PNG files.	JSON parsed; Markdown and HTML were readable; PNG was a valid image.	PASS
INPUT-01	Run `probe not-an-ip -c 1`.	Non-zero exit and a short validation error.	Non-zero exit; `[ERROR] Invalid IPv4 address: 'not-an-ip'.`	PASS
CLEAN-01	Repeat the demo and inspect services.	Second run succeeds and no listeners remain.	Repeat succeeded; ports were released; demo files were restored.	PASS

ID	Description	Expected result	Actual result	Status
REAL-01	Run the four live scenarios on Mininet with Ryu.	Healthy, degraded, failed, and recovery verdicts with evidence.	Completed on 5 Nov 2026. Degraded: 8.00% loss, 66.33 ms, 95% localization from 17 qdisc drops. Recovery: 0% loss on the backup path.	PASS

19. Test Execution Summary

Item	Value
Test cases designed	17
Test cases executed	17
Test cases passed	17
Test cases failed	0
Test cases not run at the end	0
Success rate	100%
Automated unit and integration tests	82 passed
Total measured coverage	68%
Bugs found and fixed	1 (reroute destination address)
Unresolved bugs	0

The live measurements from the latest four-scenario run are:

Scenario	Probes (sent / received)	Loss	Mean RTT	Result
Healthy	50 / 50	0.00%	9.68 ms	`HEALTHY_PATH`
Degraded	100 / 92	8.00%	66.33 ms	`DEGRADED_PATH`
Failed	20 / 0	100.00%	–	`FAILED_PATH`
Recovery before reroute	20 / 0	100.00%	–	`FAILED_PATH`
Recovery after reroute	50 / 50	0.00%	14.29 ms	`HEALTHY_PATH`

The degraded run localized `s1 <---> s2` at 95.0% using 17 kernel qdisc drops (10 on `s1-eth2` and 7 on `s2-eth2`). The earlier run on the same day independently produced the same 95.0% result from 19 drops.

20. Conclusion

All planned test cases were executed and passed. The automated suite is green, the localhost demo completes end to end, and the four live scenarios produce the expected verdicts with saved evidence.

The tool does what the project set out to do: it measures delay and loss, discovers the SDN topology, finds the active path from the flow tables, identifies the failed link, and restores connectivity over the backup route. The remaining limitations are stated openly: IPv4 only, one controller, one small topology, and no production-hardware testing.

21. Approvals

Name	**Role**	**Signature / Date**
Sudhanva Y K	Developer	
Suhana A Ghosh	Developer	
Siddhanth S	Tester	
Project Guide	Reviewer	